UniversitateaTehnică“GheorgheAsachi”dinIaşi

FACULTATEADEAUTOMATICĂȘICALCULATOARE

ELECTRONICĂ DIGITALĂ

Proiect

Tema: COUNTER BCD–v3

Studenţi: Ghiorghiu Teodor

Grupa:1212A

Coordonator: Asist.Drd.IonicaPletea

2024

Tema proiectului:

COUNTER BCD–v3

- **Specificaţiile proiectului:**

COUNTER-BCD – v3
Să se implementeze în FPGA prin descriere în limbaj VHDL un sistem secvențial de 4 biți: cu reset prioritar activ pe 0 cu două intrări de selecție din care să se stabilească funcționarea de registru serie, paralel sau numărător BCD direct. Frecvența clock-ului va fi de 1 Hz. Afișarea se va face pe display-ul cu 7 segmente.

Fișierul bitstream rezultat în urma procesului de implementare va fi verificat utilizând placa de dezvoltare BASYS3.

- **Modulul COUNTER BCD–v3**

Pentru a crea un counter BCD, am folosit semnalul creat de clock-ul intern al plăcuței FPGA, pe care l-am divizat astfel încât să obținem o frecvență de 1 Hz, adică o perioadă de 1 secundă. De asemenea, am implementat și un reset prioritar activ pe „0”, a cărui valoare este preluată de la intrare, conectată la un switch. Counter-ul folosește un semnal logic standard utilizat pentru a număra în cod binar, precum și un set de date pe 4 biți preluat de la switch-uri, care se va păstra în registrul paralel, și un semnal pentru registrul serie.

Afișarea se va face pe display-ul cu 7 segmente pentru numărător și registrul paralel și pe LED-uri pentru vizualizarea funcționării registrului serie.

Date de intrare:

clk–STD_LOGIC reprezintă ceasul extern al plăcuței

reset - in STD_LOGIC reprezintă switch-ul U1 de pe plăcuță cu ajutorul căruia vom selecta voloarea pentru reset.

data_in - in STD_LOGIC reprezintă switch-ul W2 de pe plăcuță cu ajutorul căruia vom selecta voloarea pentru registrul serie.

sel -in STD_LOGIC_VECTOR (1 downto 0) reprezinta switch-urile T1, R2 de pe plăcuță cu ajutorul căruia vom selecta modul de functionare: numarator sau registru;

D - in STD_LOGIC_VECTOR (3 downto 0) reprezinta switch-urile V16, V17, W16, W17 care vor genera datele de intrare pentru registrul paralel;

Variabile locale:

n – de tip integer, contorizează fiecare tranziție pozitivă a semnalului clk și generează un semnal de ieșire divizat (clk_out) cu o frecvență redusă

Semnale:

clk_out – de tip std_logic, semnal intern generat de procesul de divizare a frecvenței clk și este utilizat pentru sincronizarea procesului de incrementare al numărătorului binar.

a – un vector de 4 biți de tipul std_logic_vector (3 downto 0), incrementat la fiecare tranziție pozitivă a semnalului clk_out.Va stoca valoarea numărătorului binar, care va fi afisat pe display-ul cu 7 segmente.

Date de ieșire:

led : out STD_LOGIC_VECTOR (3 downto 0)- cu ajutorul caruia vom vizualiza functionarea registrului in serie.

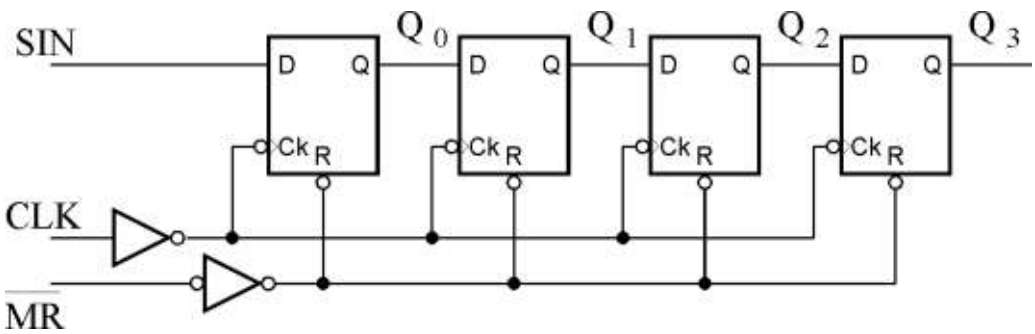
an : out STD_LOGIC_VECTOR (3 downto 0)-pentru a controla anodurile display-ului (cate cifre)

seg : out STD_LOGIC_VECTOR(6 downto 0)-pentru afisarea pe display-ul cu 7 segmente.

Regiştrii sunt circute digitale secvenţiale de stocare a informaţiei sub formă binară ce permit scrierea sau citirea simultană a tuturor biţilor.

Din punct de vedere structural, regiştri, sunt constituiţi din mai multe circute de tip flip-flop ce sunt grupate impreuna. Descrierea unui proces secvenţial in limbaj VHDL se face prin urmărirea evenimentelor unui semnal de tact (a unui semnal de ceas pe care îl vom denumi generic **clk**).

Registrul serie presupune incarcarea bit cu bit a informatiei in registru.



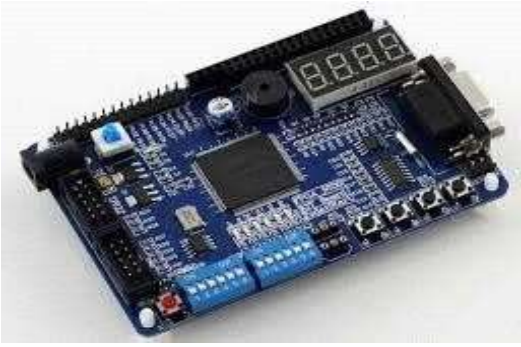
Aceste două funcții sunt implementate în librăria **std_logic_1164** – librărie ce este inclusă în mod implicit atunci când fișierul `.vhd` este generat automat.

Numărătoarele sunt circuite secvențiale de „integrare” a semnalului de clock ce se aplică la intrare. În funcție de modul de efectuare a resetului se evidențiază numărătoare cu reset asincron (atunci când semnalul de reset este prioritar față de semnalul de clock) și numărătoare cu reset sincron (resetarea are loc în situația îndeplinirii a două condiții: activarea semnalului de reset și apariția unui eveniment de clock).

- # Metodă de implementare

  În cadrul acestui proiect au fost utilizate următoarele resurse:

Circuitul FPGA xc7a35tcpg236-1 din familia Artix-7, produs de Xilinx, este utilizat prin intermediul plăcii de dezvoltare Basys3. Aceasta permite implementarea unor soluții flexibile și scalabile în domeniul ingineriei electronice. Circuitul poate fi programat pentru a îndeplini diverse funcții și algoritmi, oferind astfel un grad ridicat de flexibilitate și adaptabilitate în proiectarea sistemelor electronice.



Vivado este un instrument dezvoltat de Xilinx pentru sinteza și implementarea FPGA, care simplifică procesul de proiectare și dezvoltare a circuitelor logice programabile. Acesta include o suită completă de unelte performante pentru proiectarea, simularea, testarea și implementarea sistemelor digitale bazate pe FPGA.

- VHDL (Very High Speed Integrated Circuit Hardware Description Language) este un limbaj dedicat descrierii hardware-ului, utilizat în proiectarea și simularea circuitelor digitale. Acesta a fost creat pentru a oferi inginerilor un mijloc de a defini atât comportamentul, cât și interconexiunile componentelor dintr-un sistem electronic complex.

- # Descrierea(scurtă) a sistemului de dezvoltare BASYS3

  - BASYS 3 este un sistem de dezvoltare hardware FPGA (Field-Programmable Gate Array) produs de Digilent. Este proiectat pentru a fi un instrument versatil și accesibil pentru dezvoltarea și testarea de circuite digitale. Bazat pe FPGA-ul de la Xilinx, vine echipat cu o varietate de caracteristici și componente. Printre acestea se numără:

  - Porturi de intrare/ieșire, incluzând porturi digitale, analogice și de comunicare, oferind posibilitatea de a interacționa cu o varietate de dispozitive externe.Dispune de un ecran LCD șibutoane care faciliteazăinteracțiuneașiafișareainformațiilor.

- Conectori și interfețe de comunicare: USB, Ethernet, UART și SPI, asigurând conectivitatea cu alte dispozitive sau calculatoare.

- Sistem de alimentare integrat, care simplifică utilizarea și testarea circuitelor în diferite scenarii de alimentare.

- **Editarea fişieruluiVHDL**

Entitatea main si librăriile

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity TopModule is
    Port ( clk : in STD_LOGIC;
           reset : in STD_LOGIC;
           sel : in STD_LOGIC_VECTOR (1 downto 0);
           data_in : in STD_LOGIC;
           led : out STD_LOGIC_VECTOR (3 downto 0);
           D : in STD_LOGIC_VECTOR ( 3 downto 0 );
           seg : out STD_LOGIC_VECTOR(6 downto 0);
           an : out STD_LOGIC_VECTOR ( 3 downto 0)
);
end TopModule;
```

Codul

```
architecture Behavioral of TopModule is
    signal a : STD_LOGIC_VECTOR (3 downto 0) := "0000";
    signal shift_reg : STD_LOGIC_VECTOR (3 downto 0) := "0000";
    signal clk_out : STD_LOGIC := '0';

    begin
    an(0) <= '0';
    an(1) <= '1';
    an(2) <= '1';
    an(3) <= '1';
```

Divizorul de frecvență

```
process(clk)
    variable  n : integer range 0 to 1000000000;
    begin
    if clk'event and clk='1' then
        if n < 100000000 then
            n := n+1;
        else
            n := 0;
        end if;

        if n <= 50000000 then
            clk_out <= '1';
        else
            clk_out <= '0';
        end if;
    end if;
end process;
```

Numărător sau registru

```
51 process(clk, reset)
52     begin
53         if reset = '0' then
54             a <= "0000";
55             shift_reg <= "0000";
56         elsif rising_edge(clk) then
57             case sel is
58                 when "01" => -- BCD counter
59                     if a="1001" then
60                        a <= (others => '0');
61                     else
62                         a <= a + 1;
63                     end if;
64                 when "10" => -- shift register
65                     shift_reg <= shift_reg(2 downto 0) & data_in;
66                 when "11" => -- paralel
67                     shift_reg <= D;
68                 when others =>
69                     null;
70             end case;
71         end if;
72     end process;
73
74     led <= shift_reg;
```

Afișarea pe display

```
    seg(0) <= (a(2) and not(a(1)) and not(a(0))) or (not(a(3)) and not(a(2)) and not(a(1)) and a(0));
    seg(1) <= (a(2) and not(a(1)) and a(0)) or (a(2) and a(1) and not(a(0)));
    seg(2) <= (not(a(2)) and a(1) and not(a(0)));
    seg(3) <= (a(2) and not(a(1)) and not(a(0))) or (a(2) and a(1) and a(0)) or (not(a(3)) and not(a(2)) and not(a(1)) and
    a(0));
    seg(4) <= a(0) or (a(2) and not(a(1)));
    seg(5) <= (not(a(3)) and not(a(2)) and a(0)) or (a(1) and a(0)) or (not(a(3)) and not(a(2)) and a(1));
    seg(6) <= (not(a(3)) and not(a(2)) and not(a(1))) or (a(2) and a(1) and a(0));
end Behavioral;
```

- **Editarea fișierului de constrângeri**

```
## Clock signal
set_property PACKAGE_PIN W5 [get_ports clk]

set_property IOSTANDARD LVCMOS33 [get_ports clk]


## Switches
set_property PACKAGE_PIN U1 [get_ports reset]
set_property IOSTANDARD LVCMOS33 [get_ports reset]
set_property PACKAGE_PIN T1 [get_ports {sel[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {sel[0]}]
set_property PACKAGE_PIN R2 [get_ports {sel[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {sel[1]}]
set_property PACKAGE_PIN W2 [get_ports data_in]
set_property IOSTANDARD LVCMOS33 [get_ports data_in]
set_property PACKAGE_PIN V17 [get_ports {D[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {D[0]}]
set_property PACKAGE_PIN V16 [get_ports {D[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {D[1]}]
set_property PACKAGE_PIN W16 [get_ports {D[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {D[2]}]
set_property PACKAGE_PIN W17 [get_ports {D[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {D[3]}]

## LEDs
set_property PACKAGE_PIN U16 [get_ports {led[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {led[0]}]
set_property PACKAGE_PIN E19 [get_ports {led[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {led[1]}]
set_property PACKAGE_PIN U19 [get_ports {led[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {led[2]}]
set_property PACKAGE_PIN V19 [get_ports {led[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {led[3]}]
```

```
set_property PACKAGE_PIN W6 [get_ports {seg[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {seg[1]}]
set_property PACKAGE_PIN U8 [get_ports {seg[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {seg[2]}]
set_property PACKAGE_PIN V8 [get_ports {seg[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {seg[3]}]
set_property PACKAGE_PIN U5 [get_ports {seg[4]}]
set_property IOSTANDARD LVCMOS33 [get_ports {seg[4]}]
set_property PACKAGE_PIN V5 [get_ports {seg[5]}]
set_property IOSTANDARD LVCMOS33 [get_ports {seg[5]}]
set_property PACKAGE_PIN U7 [get_ports {seg[6]}]
set_property IOSTANDARD LVCMOS33 [get_ports {seg[6]}]
#set_property PACKAGE_PIN V7 [get_ports dp]
#set_property IOSTANDARD LVCMOS33 [get_ports dp]
set_property PACKAGE_PIN U2 [get_ports {an[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {an[0]}]
set_property PACKAGE_PIN U4 [get_ports {an[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {an[1]}]
set_property PACKAGE_PIN V4 [get_ports {an[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {an[2]}]
set_property PACKAGE_PIN W4 [get_ports {an[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {an[3]}]
```

- **Descrierea paşilor de sinteză şi testarea circuitului rezultat**
  - Sinteză:
    - Crearea unui proiect în Vivado și importarea fișierelor VHDL ale circuitului.

    - Adăugarea fișierului de constrângeri .xdc la proiect pentru a asocia semnalele circuitului cu pini specifici ai plăcii de dezvoltare.

- Implementare:
    - Verificarea și optimizarea rulării pentru a asigura o propagare adecvată a semnalelor și o performanță optimă..
    - Generarea fișierului bitstream, care conține configurația FPGA pentru circuitul sintetizat.
- Programarea FPGA:
    - Utilizarea programului Vivado pentru a încărca fișierul bitstream în placa FPGA..
- Testarea în simulare:
    - Crearea testbench-ului în Vivado pentru a simula comportamentul circuitului.
    - Definirea semnalelor de intrare și verificarea rezultatelor la semnalele de ieșire așteptate.
    - Executarea simulării și analizarea rezultatelor pentru a verifica corectitudinea funcțională a circuitului.
- Testarea pe placă FPGA:
    - Încărcarea fișierului bitstream în placa FPGA și verificarea comportamentului circuitului în cadrul plăcii.
    - Utilizarea instrumentelor pentru depanare și verificarea semnalelor pentru a valida funcționarea corectă a circuitului pe placa FPGA.

• Concluzii

In concluzie, am realizat in FPGA un sistem secvential de 4 biti cu reset prioritar active pe ‘0’ cu 2 intrari de selectie care stabilesc functionarea de registru series au numarator BCD direct, cu afisaj pe displayul 7 segmente, in limbaj VHDL.

Bibliografie:

- VHDLReferenceManual,http://www.ics.uci.edu/~jmoorkan/vhdlref/Synario%20VHDL%20Manual.pdf
- BASYS3ReferenceManual,https://reference.digilentinc.com/reference/programmable-logic/basys- 3/reference-manual